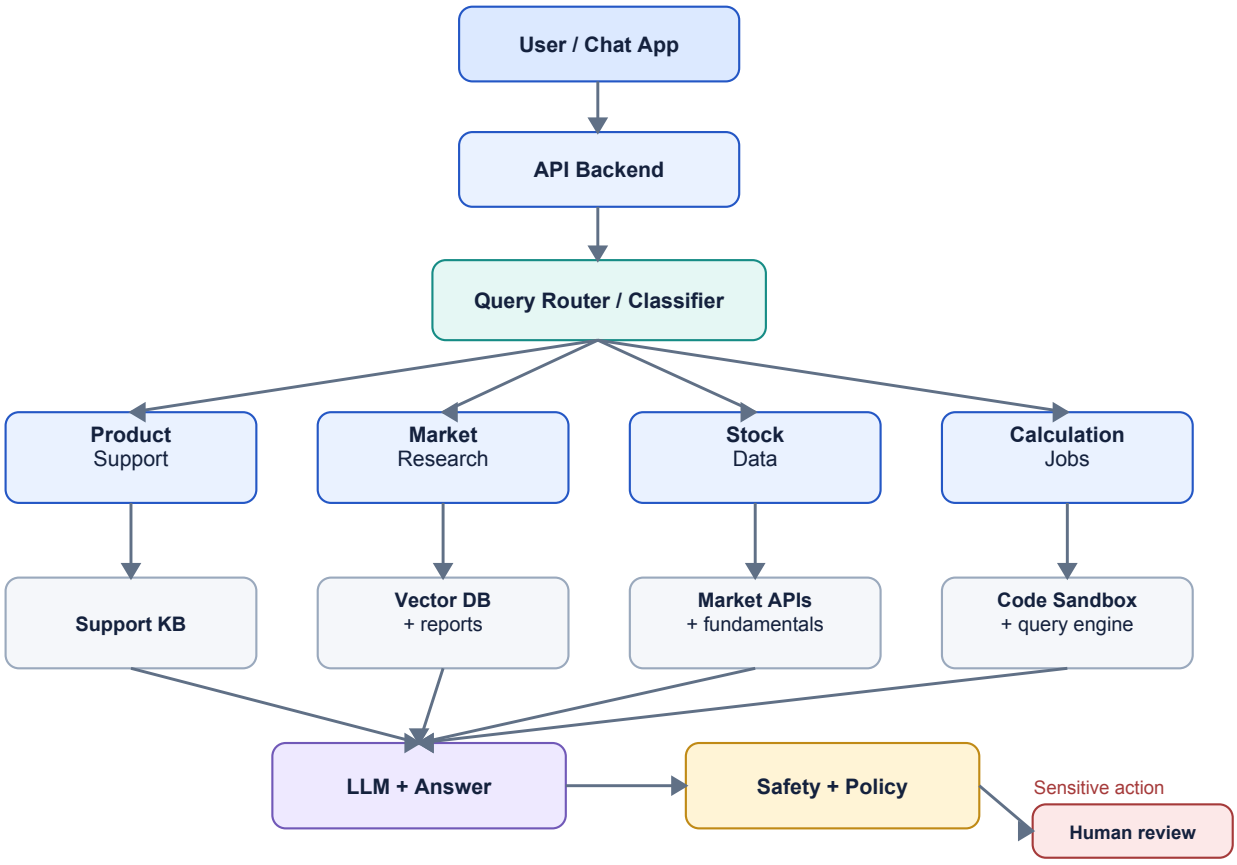
FINANCIAL CHAT SYSTEM DESIGN

Relearning Roadmap

A beginner-friendly study guide for rebuilding the software, data, cloud, and AI foundations behind a production financial chat application.



The destination

Explain what every component does, why it exists, how it communicates, what can fail, and why this design is preferable to the alternatives.

Study-guide edition | Built from the complete learning plan

1. What the assignment is actually testing

You do not need to relearn your entire Electronic and Computer Engineering diploma first. This project is mainly about software systems, backend architecture, databases, cloud infrastructure, APIs, and modern AI systems.

Imagine one brokerage chat box receiving all of these messages:

```
"What is Apple's current share price?"
"Summarize the latest analyst reports about NVIDIA."
"How do I request a refund?"
"Using our historical dataset, calculate Tesla's 30-day volatility."
"Refund my subscription."
```

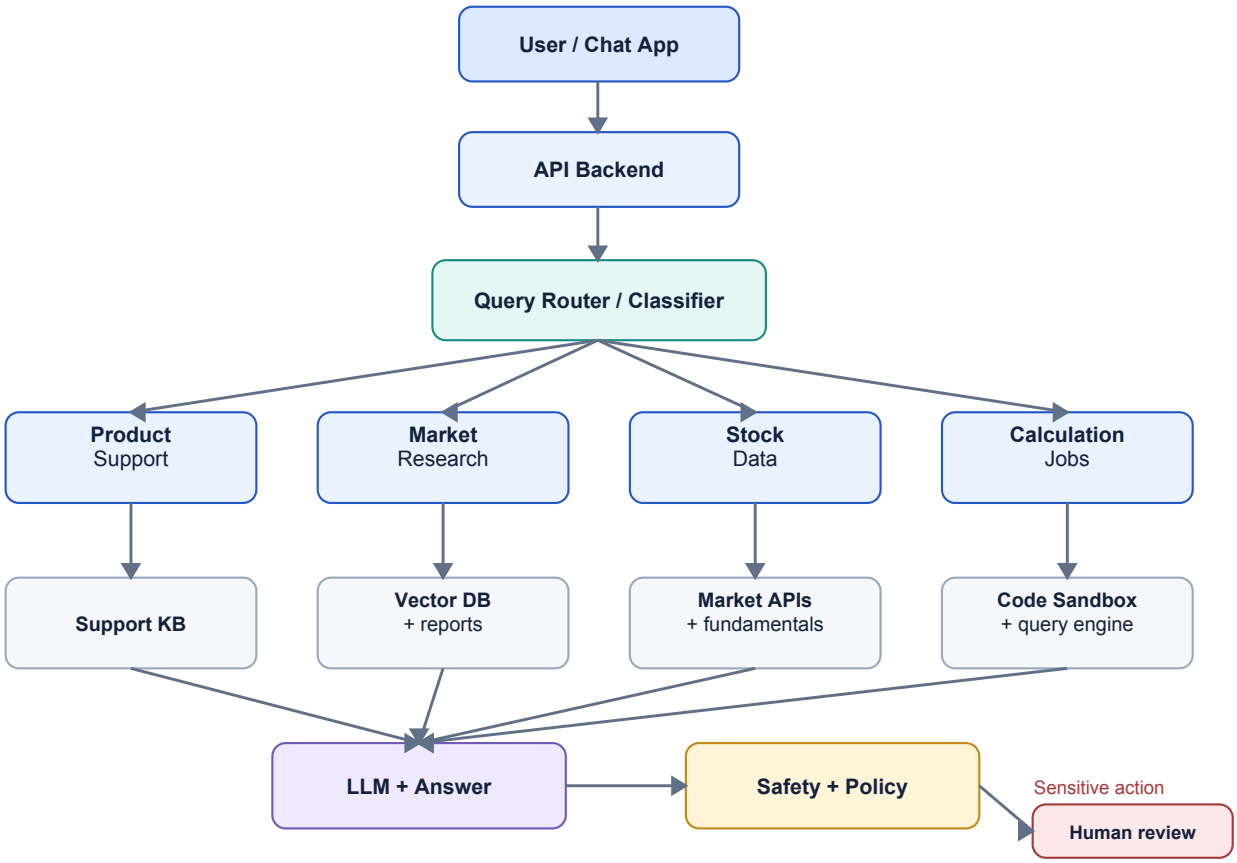
They look alike because they are all chat messages, but each requires a different technical path.

The real question is:

> How do you build the system behind the chat box so it understands intent, finds the right information, performs calculations, responds quickly, survives heavy traffic, and prevents unsafe actions?

2. The big picture

The architecture is a map, not a list to memorize. Use it to connect each requirement to a component.



A query first reaches the backend and router. Product questions use support knowledge; research questions search fresh reports; stock questions use appropriate data sources; calculation requests run in an isolated sandbox. The LLM assembles a grounded answer, while policy checks intercept sensitive actions and route

them to a human.

Your system-design goal

For every box, be ready to explain its responsibility, input and output, failure modes, scaling strategy, security boundary, and trade-offs.

3. Layer 1 - Basic software and computing foundations

Rebuild the mental model of how software works before diving into AI or cloud architecture.

- Client vs server; frontend vs backend
- HTTP and HTTPS; requests and responses
- APIs, REST, and JSON
- Authentication vs authorization
- Processes, CPU, memory, and storage
- IP addresses, DNS, ports, TCP, and latency

```
Phone -- HTTPS request --> Backend server -- SQL query --> Database
```

Checkpoint

Explain, in plain language, what roughly happens between pressing Send in a chat app and receiving the first words of the reply.

4. Layer 2 - Programming with Python

Python is the most useful primary language for this project. Start with variables, data types, conditions, loops, functions, lists, dictionaries, classes, exceptions, files, modules, and packages.

Then connect those basics to real systems:

- HTTP requests and APIs
- JSON
- SQL and Pandas
- Asynchronous programming
- Basic backend development

```
def get_stock_price(symbol):
price = stock_api.get(symbol)
return price

@app.get('/stock/{symbol}')
def stock(symbol):
return get_stock_price(symbol)
```

The important step is seeing how a small function becomes part of a backend endpoint that another application can call.

5. Layer 3 - Databases and data engineering

The 500 GB internal-data requirement is a deliberate clue: the system cannot load everything into Python memory.

Begin with database fundamentals:

- Tables, rows, columns, primary keys, and foreign keys
- SELECT, WHERE, JOIN, GROUP BY, and indexes
- Relational data and when SQL fits
- Object storage, data warehouses, and data lakes
- Caching, indexing, partitioning, and replication

```
SELECT price
FROM stocks
WHERE symbol = 'AAPL';

500 GB dataset -> durable storage -> query engine -> required rows only -> Python sandbox
```

For a request such as calculating AAPL's average return from 2020 to 2025, the query engine should retrieve only the relevant symbol, dates, and columns. This keeps transfer, memory, and compute manageable.

Design principle

Move the computation close to the data when practical, and move only the smallest useful result into the sandbox.

6. Layer 4 - Backend system design

This layer joins the application pieces into services that can handle traffic and failure.

- Servers, APIs, and service boundaries
- Load balancers and horizontal vs vertical scaling
- Caches, queues, and background workers
- Rate limits, timeouts, retries, and circuit breakers
- Health checks, logs, metrics, and alerts

```
Users -> Load balancer -> [Server A | Server B | Server C]
```

If traffic grows from 100 to 10,000 requests per second during a market event, multiple stateless application servers can share the load. Queues can separate expensive calculation jobs so they do not overwhelm fast chat requests.

Requirement connection

Elastic load handling means scaling capacity, isolating heavy work, applying backpressure, and degrading gracefully when dependencies are overloaded.

7. Layer 5 - AI and LLM fundamentals

Only after the base layers should you go deeper into the AI-specific pieces.

- LLMs, tokens, context windows, prompts, and system prompts
- Hallucination and temperature
- Embeddings, semantic search, and vector databases
- Retrieval-Augmented Generation (RAG)
- Tool or function calling, agents, and structured outputs

RAG is central because the corpus contains more than 5,000 reports and grows by about 100 per day. You cannot paste the entire corpus into every prompt.

```
Reports -> split into chunks -> create embeddings -> vector database

Question -> query embedding -> vector search -> relevant passages -> LLM -> grounded answer
```

For a question about NVIDIA's AI revenue, retrieval finds the most relevant report passages. The LLM answers using that evidence and should include source metadata or citations so users can verify it.

Freshness pipeline

Ingest new reports, extract text, clean and chunk them, attach metadata, create embeddings, update the index, and monitor failures. Freshness is an operational pipeline, not a one-time upload.

8. Layer 6 - Routing and orchestration

The system should not treat every message identically. It must classify intent, select tools, and coordinate steps.

```
"How do I change my password?" -> PRODUCT SUPPORT
"What is Tesla trading at?" -> LIVE STOCK DATA
"What are analysts saying about Tesla?" -> MARKET RESEARCH / RAG
"Calculate Tesla's volatility over 3 years." -> CALCULATION / SANDBOX
```

- Classification: what kind of request is this?
- Routing: which subsystem should handle it?
- Tool selection: which API, database, or retrieval system is appropriate?
- Orchestration: what steps run, in what order, and which can run in parallel?

Practical trade-off

Use deterministic rules for high-risk boundaries, a small fast classifier for common intent routing, and the larger LLM where language understanding or synthesis adds value.

9. Layer 7 - Reliability, performance, and distributed systems

The target is an average time to first token (TTFT) below two seconds. That is the delay before the user sees the first generated text.

```
1 second: fast
2 seconds: acceptable target
5 seconds: slow
```

```
15 seconds: appears broken
```

A heavy path may include classification, database lookup, vector search, an external API call, the LLM, and code execution. Useful techniques include:

- Stream the answer as soon as generation begins
- Cache safe, reusable results with appropriate freshness limits
- Run independent retrievals in parallel
- Use async I/O and connection pooling
- Precompute expensive reusable values
- Use a smaller model for simple classification
- Set timeouts, retry selectively, and provide fallbacks

```
Sequential: A -> B -> C -> D
Parallelizable: A -> [B and C] -> D
```

Do not force every workload into the same latency promise. A stock quote should be fast; a large computation can acknowledge the job quickly, stream status, and finish asynchronously.

10. Layer 8 - Security and AI safety

A financial application must assume sensitive data, valuable actions, and adversarial input.

- Authentication, authorization, and role-based access control
- TLS, encryption, and secrets management
- Input validation, audit logs, rate limiting, and least privilege
- Prompt injection defenses and data-leakage controls
- Personally identifiable information (PII)
- Sandboxing and human-in-the-loop approval

The AI may explain the refund policy, but it must not issue a refund by itself.

```
User request -> intent detection -> REFUND ACTION -> policy and permission check -> human review ->
approve or reject -> refund system
```

Core principle

An LLM's ability to understand an instruction does not justify giving it unlimited authority to execute that instruction.

11. Financial concepts to learn

You do not need investment-banking expertise, but you do need enough domain vocabulary to choose correct data sources and interpret questions.

- Stocks, shares, ticker symbols, market price, bid and ask
- Market capitalization, revenue, profit, EPS, P/E, and dividends

- Financial statements, returns, volatility, and trading volume
- Historical data vs real-time or near-real-time market data

Fundamental data	Market data
Revenue, profit, EPS, P/E, balance sheet, cash flow	Current price, bid, ask, volume, daily high/low, price history

This distinction matters because live prices usually come from a market-data provider with freshness and entitlement constraints, while company fundamentals may come from filings or a financial-data store.

12. Docker and sandboxing

Generated code must run inside a temporary isolated environment, never directly on production servers.

- Process and container isolation
- CPU and memory limits
- Execution timeouts
- Restricted network access
- Read-only or disposable filesystems
- Allowlisted libraries and data-access interfaces
- Output size limits and audit logs

```
LLM -> generated Python -> isolated container
CPU: limited
RAM: limited
runtime: limited
network: restricted
filesystem: isolated
-> sanitized result
```

These controls contain infinite loops, excessive memory use, dangerous file operations, and unauthorized network access.

13. The 14-phase learning roadmap

Do not try to learn everything simultaneously. Progress in order, but overlap adjacent phases and build small projects throughout.

Phase	Learn	Why
1	Computer and software fundamentals	Rebuild your mental model
2	Python	Learn to build things
3	Networking, HTTP, and APIs	Understand system communication
4	SQL and databases	Understand data
5	Backend development	Build services
6	Git, Linux, and Docker	Use a real engineering workflow
7	System-design fundamentals	Understand scalable architecture
8	LLM fundamentals	Understand the AI

Phase	Learn	Why
9	Embeddings, vector DB, and RAG	Build knowledge retrieval
10	Tool calling and routing	Build AI workflows
11	Distributed systems and scaling	Handle real traffic
12	Security and reliability	Make it production-worthy
13	Financial-data basics	Understand the domain
14	Build the project	Combine everything

Important

You do not need to master Phase 1 before touching Phase 2. Learning becomes stickier when theory and small builds reinforce each other.

14. How engineers translate requirements into design questions

The deepest lesson is not drawing a beautiful diagram. It is learning to break vague requirements into concrete engineering problems and trade-offs.

Requirement	Engineering questions it should trigger
500 GB internal dataset	Where is it stored? How is it partitioned and queried? How much data does each job really need? What happens under concurrent load?
TTFT below 2 seconds	Where is latency spent? What can be cached, parallelized, precomputed, streamed, or handled by a smaller model?
100 new reports per day	How are documents ingested, parsed, chunked, embedded, indexed, monitored, and made searchable quickly?
Refunds need human approval	How do we distinguish information from action? Where is the action intercepted? Who approves it? What authority does the AI have?
Sudden traffic spike	Can services scale horizontally? Do heavy jobs need a separate queue? How do we protect lightweight requests and apply backpressure?

That habit - asking where data lives, where latency comes from, where authority is checked, and where failure is contained - is the heart of system design.

15. Recommended staged learning approach

Stage 1: How does a computer program work?

Build: Write small Python programs.

Stage 2: How do two computers communicate?

Build: Call a public HTTP API.

Stage 3: How does a backend/API work?

Build: Build a tiny stock endpoint.

Stage 4: How does a database work?

Build: Build a small stock database.

Stage 5: How do large systems scale?

Build: Add caching, a queue, and multiple workers.

Stage 6: How does an LLM work?

Build: Experiment with prompts and structured outputs.

Stage 7: How does RAG work?

Build: Build a mini report-search system.

Stage 8: How does an AI use tools?

Build: Route questions to support, stock, research, or calculation.

Stage 9: How do we make it safe and reliable?

Build: Add permissions, sandbox limits, timeouts, fallbacks, and monitoring.

Stage 10: Build the financial chat system

Build: Combine the pieces and defend the trade-offs.

Why this works

By the final stage, the architecture is not a diagram you memorized. You understand why each component exists because you encountered the problem that required it.

Your first practical study cycle

Use this guide as a loop: learn a concept, build the smallest useful example, explain it without jargon, then connect it back to the financial-chat requirements.

Suggested first four builds

- A Python script that fetches a stock quote from an API and prints JSON
- A small backend with one stock endpoint and one support endpoint
- A SQL database containing symbols and historical prices, queried by date range
- A rule-based router that labels sample messages as PRODUCT, STOCK, MARKET, or CALCULATION

For each build, write down:

- What problem does this solve?
- What are the inputs and outputs?
- What can fail?
- How would it behave with 1,000 times more traffic or data?
- What security boundary is required?
- What alternative design did you reject, and why?

Final perspective

You are rebuilding your technical foundation through one coherent final project. Progress comes from repeatedly connecting a small piece of working software to the larger architecture - not from memorizing every term before you begin.